

第9章 流量控制

流量控制（Flow Control）的概念起源于网络通信。一个复杂的网络系统由各类设备（如交换机、路由器、核心网），与这些设备之间的连接通路组成。从数据传输的角度来看，整个网络中具有两类资源，一类是数据通路，另一类是数据缓冲。

数据通路是网络上最珍贵的资源，直接决定了数据链路可能的最大带宽；而数据缓冲是另外一个重要的资源。当一个网络上的设备从一点传送到另外一点时，需要通过网络上的若干节点才能最终到达目的地。在这些网络节点中含有缓冲区，暂存在这个节点中没有处理完毕的报文。网络设备使用这些数据缓冲，可以搭建数据传送的传送流水线，从而提高数据传输性能。最初在网络设备中只为一条链路提供了一个缓冲区，如图 9-1 所示。

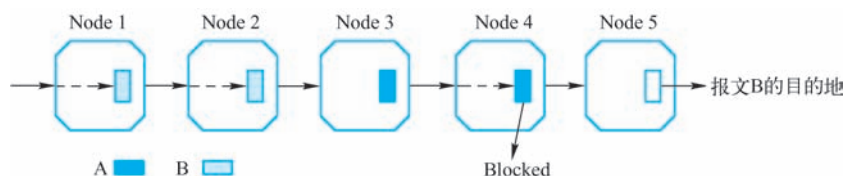


图 9-1 基于单数据通路的数据传递

当网络设备使用单数据通路进行数据传递时，假设在该通路中正在传递两个数据报文，分别是 A 和 B。其中数据报文 B 需要经过 Node1~5 才能到达最终的目的地，而数据报文 B 在经过 Node 3 时发现由于 Node 3 正在向 Node 4 发送一个数据报文 A。从而数据报文 B 到达 Node 3 后，由于 Node 4 的接收缓存被数据报文 A 占用，而无法继续传递。此时虽然在整个数据通路中，Node 4 和 Node 5 之间的通路是空闲的，但是报文 B 还是无法通过 Node 3 和 4，因为在 Node 4 中只有一个数据缓冲，而这个数据缓冲正在被报文 A 使用。

使用这种数据传送规则会因为一个节点的数据缓冲被占用，而影响了后继报文的数据传递。为了解决这个问题，在现代网络节点中设置了多个虚通路 VC，不同的数据报文可以使用不同的通路进行传递。从而有效解决了单数据通路带来的问题，基于双数据通路的数据传递如图 9-2 所示。

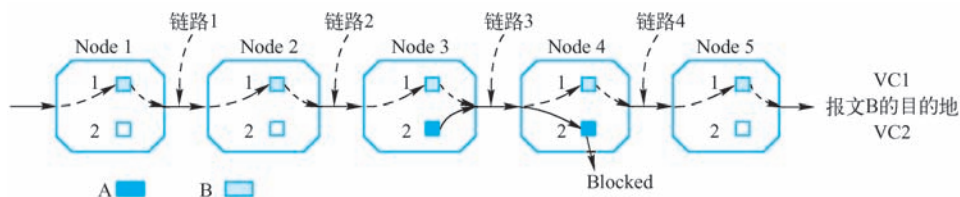


图 9-2 基于双数据通路的数据传递

如上图所示，所谓多通路是指在每一个节点内设置两个以上的缓存。上例中设置了两个缓存，报文 A 经过 Node 1~5 时使用缓存 2 进行缓存，然后进行数据传递，而报文 B 使用缓存 1 进行缓存。因此虽然报文 A 因为某种原因不能继续传递，也只是将报文阻塞在缓存 2 中，而不影响报文 B 的数据传递。

所谓 VC 是指缓存到缓存之间的传递通路。如图 9-2 所示的例子中含有两个 VC，分别是 VC1 和 VC2。其中 VC1 对应节点间缓存 1 到缓存 1 的数据传递，而 VC2 对应缓存 2 到缓存 2 的数据传递。VC 间的数据传递，如 Node 1 的缓存 1/2 到 Node 2 的缓存 1/2，都要使用实际的物理链路“链路 1”，这也是将 VC 称为“虚”通路的主要原因。

在一个实际的系统中，虚通路的使用需要遵循一定的规则。如在 PCIe 总线中，将不同的数据报文根据 TC 分为 8 类，并约定这些 TC 报文可以使用哪些 VC 进行数据传递。在 PCIe 总线中使用 TC/VC 的映射表，决定 TC 与 VC 的对应关系。

PCIe 总线规定同一类型的 TC 报文只能与一条 VC 对应，当然从理论上讲，不同的 TC 报文可以与不同的 VC 对应，也可以实现一种自适应的算法根据实际情况实现 TC 报文和 VC 的对应关系。只是使用这种方法需要付出额外的硬件代价，效果也不一定明显。

下文以图 9-2 所示的数据传递为例，进一步对此说明相同类型的报文使用不同 VC 的情况。假设报文 A 和 B 属于相同种类的报文，但是报文 A 使用 VC1，而报文 B 使用 VC2。首先报文 A 传递到 Node 4 后被阻塞。而报文 B 使用的 VC 和报文 A 使用的 VC 不同，报文 B 最终也会到达 Node 4。

由于报文 A 和报文 B 属于相同类型的报文，Node 4 阻塞报文 B 的概率非常大，因为 Node 4 已经阻塞了报文 A。阻塞报文 A 的原因在很大概率上也会对报文 B 适用。此时两个虚通路都被同一种类型的报文阻塞，其他报文将无法通过。因此在实际应用中，相同类型的数据报文多使用同一个 VC 进行数据传递，而在 PCIe 总线中，一个 TC 只能对应一个 VC。

目前多通路技术的应用已经普遍应用到网络传输中，虚通路是一种防止节点拥塞的有效方法。但是在网络传送中，还存在一种不可避免的拥塞现象，即某一条链路或者某个节点是整个系统的瓶颈。

我们假设图 9-2 中 Node 4 将报文转发到 Node 5 的速度低于 Node 3 发送报文的速度。在这种情况下，Node 4 将成为整个传送路径上的瓶颈，无论 Node 4 中的缓存 1 和 2 有多大，总会被填满，从而造成节点拥塞。

当缓存填满后，如果 Node 3 继续向 Node 4 发送报文时，Node 4 将丢弃这些报文，之后 Node 3 将会择时重发这个报文，而 Node 4 仍然会继续丢弃这个报文，这种重复丢弃的行为将极大降低网络带宽的利用率，而且 Node 3 也会成为网络中新的瓶颈，从而引发连锁反应，造成整个网络的拥塞。为了避免这类事件发生，网络中的各个组成部件需要对数据传送进行一定的流量控制，合理地接收和发送报文。

如上文所述，在网络中有两类资源，一类是数据通路，另一类是数据缓冲。而流量控制的作用是合理地管理这两类资源，使这些资源能够被有效利用。

9.1 流量控制的基本原理

目前流量控制从理论到实现大多基于多通道技术，本节也仅讨论基于多通道的流量控制（FCVC，Flow-Controlled Virtual Channels）的基本原理。

流量控制的主要作用是在发送端和接收端进行数据传递时，合理地使用物理链路，避免因接收端缓冲区容量不足而丢弃来自发送端的数据，从而要求发送端重新发送已经发送过的报文，并最终有效地利用网络带宽。

目前几乎所有流量控制算法的核心都是根据接收端缓冲区的容量，向发送端提供反馈。而发送端根据这个反馈，决定向接收端发送多少数据。这些流量控制算法都力求发送的每一个数据报文都能够被接收端正确接收，而不会被接收端因为缓冲不足而丢弃。使用流量控制机制并不能提高网络的峰值带宽，相反还会降低网络的带宽，但是可以有效减少数据报文的重新发送，从而保证网络带宽被充分利用。

流量控制的目标是在充分利用网络带宽的前提下，尽量减少数据报文因为接收端缓存容量不足而被丢弃的情况，因为此时数据发送端将会择时重新传送这些丢弃的报文，从而降低了数据通路的利用率。

为了实现流量控制，数据接收端需要及时地向发送端反馈一些信息，发送端依此决定，是否能够向接收端继续发送数据。这些反馈信息也需要占用一定的数据通路带宽。但是采用这种方法可以有效避免数据报文的丢失与重发，从而在整体上提高了数据通路的利用率。流量控制针对端到端的数据传递，目前流行的流量控制方法共有两种，分别为 Rate-Based 机制和 Credit-Based 机制。

9.1.1 Rate-Based 流量控制

Rate-Based 机制适合“可预知带宽”的数据传递方式，而 Credit-Based 机制更加适合“突发数据传送”。下面将以图 9-3 所示的实例简单介绍 Rate-Based 机制。

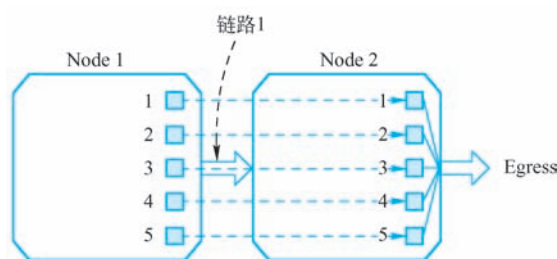


图 9-3 Rate-Based 流量控制机制

假设 Node 1 与 Node 2 之间共存在 5 个 VC，即在链路 1 的两端设置了 5 组缓存。而这 5 个 VC 共享一个物理通路，即链路 1。为方便起见，假设链路 1 的带宽为 1，而在系统初始化时，将 VC1~5 可以使用的带宽 BCVC 都设为 $1/4$ （即 Rate 值为 $1/4$ ），而且每一个 VC 使用的最大数据传送率不能超过 BCVC。

在某些情况下，由于在 Node 1 中，每个 VC 的 BCVC 最大值为 $1/4$ ，因此 Node 1 可以向 Node 2 发送的数据带宽总和为 $5/4$ ，大于链路 1 所能提供的峰值带宽，因此链路 1 将可能成为瓶颈，从而造成网络拥塞。此时来自 Node 1 的数据报文必然会阻塞在各自 VC 的发送缓冲中，并可能出现报文重传现象。

Rate-Based 机制使用“自适应”调解的办法有效防止了这种网络拥塞。Rate-Based 机制规定每一个 VC 在发送一定数量的报文后，将主动地将相应 VC 的 Rate 调整为 Rate 减去 ADR（Additive Decrease Rate），直到 Rate 等于 MCR（Minimum Cell Rate）^①；当 Node 2 的

① 在本节的实例中， $5 \times \text{MCR} \leq 1$ 。即所有 VC 使用的 MCR 之和小于或等于链路总带宽。

Egress端口并不拥塞时，Node 2 将向 Node 1 的对应 VC 发出正向反馈，通知该 VC 可以适当提高数据传送率，当 Node 1 的 VC 收到这个正向反馈后，将更新其 BCVC。

假设在本例中 MCR 为 x ，当链路 1 严重拥塞时，Node 2 不会向 Node 1 的所有 VC 发出正向反馈，最终 Node 1 所有 VC 的 Rate 都将降为 MCR，此时 Node 1 将不会向 Node 2 发送过多的数据报文；当链路 1 并不拥塞时，Node 2 将向 Node 1 的相应 VC 发出正反馈，通知 Node 1 可以适当提高数据报文的数据传送率。

Rate-Based 流量控制机制可以使用漏桶（Leaky Bucket）算法或者令牌桶（Token Bucket）算法实现。使用令牌桶算法时，一个设备至少具有 MCR 个令牌，这个设备每发送一定数量的报文后，将令牌减少 ADR 个，但是总令牌数不低于 MCR。当这个设备收到下游设备的正反馈时，将增加令牌数。

采用 Rate-Based 流量控制机制可以有效解决“可预知带宽”的数据传递。比如 Node 1 向 Node 2 发送音频或者视频数据，这些音视频数据占用的数据带宽基本恒定，因此使用这种方法可以保证这类数据报文的流畅传递。

而对于多数长度“不可预知”的突发数据传递，该机制并不能完全适用。因为 Rate-Based 流量控制的实时性较弱，当一个 VC 需要瞬间传递大量报文时，Rate-Based 机制不能及时地为这条 VC 提供足够的数据传送率；而当一个 VC 拥塞时，也不能及时地降低数据传送率。因此使用 Rate-Based 机制并不能满足网络上突发数据传送的需要，此时需要使用 Credit-Based 机制对流量进行控制。

9.1.2 Credit-Based 流量控制

为便于 Credit-Based 流量控制机制的讨论，假设在网络中存在三类节点，分别是 Upstream 节点、Current 节点和 Downstream 节点，这些节点之间通过实际的物理链路互连，在每一个节点内部都使用两个 VC，其结构如图 9-4 所示。

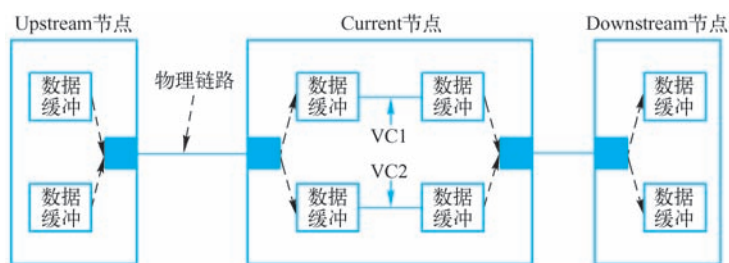


图 9-4 Upstream、Current 与 Downstream 节点的关系

其中 Upstream 节点通过物理链路将数据报文发向 Current 节点，而 Current 节点通过物理链路将数据报文发向 Downstream 节点。在虚通路的设计中，每个节点的发送端口和接收端口之间具有分属于不同 VC 的数据缓冲，这些数据缓冲之间的互连组成了不同的 VC。

Current 节点首先将来自 Upstream 节点的报文暂存在数据缓冲中，然后再发送到 Downstream 节点。当 Upstream 节点通过 Current 节点，向 Downstream 节点发送数据报文时，流量控制发生在 Upstream 节点和 Current 节点、Current 节点和 Downstream 节点之间，而不是 Upstream 节点到 Downstream 节点。简而言之，流量控制发生在链路的两端，基于端到端之

间，而不是基于节点到节点间。

在 Upstream 节点、Current 节点和 Downstream 节点中存在两个 VC，下文以其中的一个 VC 为例，说明如何使用 Credit-Based 机制进行数据传递。值得注意的是，Current 节点、Upstream 节点和 Downstream 节点只是一个相对概念。Current 节点也可以从 Downstream 节点接收数据报文，而向 Upstream 节点转发这些数据报文，从而组成一个双向通路。为简便起见，本章仅讨论在单向通路下，Credit-Based 流量控制机制的原理与实现。

Credit-Based 机制基于“Credit”进行数据传递，当 Upstream 节点需要发送数据报文时，需要具备足够的 Credit，才能向 Current 节点发送数据。这个 Credit 由 Current 节点提供，并与 Upstream 节点保存的 Credit 同步，为此 Current 节点需要定时向 Upstream 发送“传递 Credit”的数据报文。

下文为简便起见，假设节点间传送的数据报文，其长度固定，而且每次只能传递一个数据报文。Credit-Based 机制需要使用以下参数进行报文传递。

- Buf_Alloc 参数。该参数保存在 Current 节点中接收数据缓冲的总大小。Upstream 节点能够发送的数据报文总数不能大于该参数。
- Crd_Bal 参数。该参数是 Upstream 节点可以发送的数据报文数，Current 节点需要定时向 Upstream 节点发送 Credit 报文。Upstream 节点收到该报文后，使用该报文中的“Credit”同步 Crd_Bal 参数。Upstream 节点可以发送的数据报文数不能超过 Crd_Bal 参数。
- Tx_Cnt 参数。该参数为 Upstream 节点已经发送的数据报文数。
- Fwd_Cnt 参数。该参数为 Current 节点转发到 Downstream 节点的数据报文数。

Credit-Based 流量控制使用的各个参数之间的关系如图 9-5 所示。

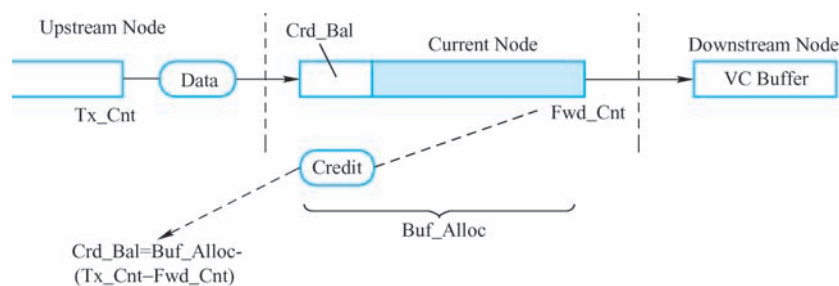


图 9-5 使用 Credit-Based 机制进行数据传递

1. Upstream 节点向 Current 节点发送报文

Upstream 节点向 Current 节点发送报文时，Current 节点必须有足够的缓冲，而且 Current 节点需要预先将其剩余的缓冲数量，即 Credit（其值为一个正整数），及时地发送给 Upstream 节点。Upstream 节点使用 Crd_Bal 参数保存这个 Credit。

Crd_Bal 参数的初始值为 Buf_Alloc，即 Current 节点能够接收的最大报文个数，该值在系统初始化时由 Current 节点发向 Upstream 节点。因此 Upstream 节点在流量控制机制初始化完毕后，Crd_Bal 参数与 Current 节点中能够存放的最大报文数相同。

Upstream 节点每成功发送一个数据报文后，Crd_Bal 值减 1，当 Crd_Bal 参数为 0 时，Upstream 节点不能向 Current 节点发送数据报文。此时 Upstream 节点必须等待 Current 节点发

送 Credit 报文，更新 Crd_Bal 参数后，才能继续发送数据报文。

2. Current 节点向 Upstream 节点发送 Credit

Current 节点收到来自 Upstream 节点的数据报文后，将会根据链路的实际情况，将这些报文转发到 Downstream 节点。

假设在一段时间之内，Current 节点共收到了 Tx_Cnt 个报文，而转发了 Fwd_Cnt 个报文，那么此时在 Current 节点中还能容纳 $\text{Buf_Alloc} - (\text{Tx_Cnt} - \text{Cx_Cnt})$ 个报文空间。Current 节点将这个值作为 Credit，发送到 Upstream 节点。而 Upstream 节点将根据这个 Credit 的值更新 Crd_Bal 参数。

3. Current 节点将报文转发到 Downstream 节点

Current 节点接收到报文之后再将其转发出去涉及一些路由算法。常用的算法有 Cut-Through 路由和 Wormhole 路由算法。

当使用 Wormhole 路由方式时，一个报文将被分解为若干个 Flit，包括 Header Flit、Data Flit 和 Tail Flit。当数据报文到达 Current 节点后，Current 节点立即对 Header Flit 进行分析，首先根据路由算法决定发向哪个 Downstream 节点^①。如果在对应的 Downstream 节点中，链路空闲且有足够的缓冲资源时，Current 节点将发送 Data Flit 直到 Tail Flit，即发送整个数据报文；如果对应的 Downstream 节点没有资源接收这个报文，数据报文将在 Current 节点中存储。

Cut-Through 路由与 Wormhole 路由类似。采用 Cut-Through 路由时，Downstream 节点必须具有接收整个报文的能力时，才能接收报文；而采用 Wormhole 路由时，Downstream 节点具有接收部分报文的能力时，就可以接收报文。采用 Wormhole 路由的优点是数据报文传送延时较短，每一个节点所需要的数据缓存相对较小。当网络发生拥塞时，采用 Wormhole 路由技术可能会使一个报文分别缓冲在 Current 节点和 Downstream 节点中，而使用 Cut-Through 路由技术数据报文最终缓冲在一个节点中。

巨型机一般使用 Wormhole 技术进行报文传递，而网络系统中多使用 Cut-Through 路由技术。有关 Wormhole 和 Cut-Through 技术的优劣分析超出了本书的范围，本书不会对此进行详细分析。巨型机应用针对的是可预知的网络拓扑结构，而网络系统的拓扑结构是变化的。在一个网络拓扑结构可预知的前提下，采用 Wormhole 技术可以在避免拥塞的前提下，降低网络报文的传递延时；而对于一个未知的网络拓扑结构，使用 Cut-Through 技术更为合理。

9.2 Credit-Based 机制使用的算法

在第 9.1.2 节中提到的 Credit-Based 机制是一个较为理想的模型，在这个模型中，没有考虑网络的延时和拥塞，也没有考虑 Current 节点何时采用哪种策略将 Credit 传送给 Upstream 节点，同时也没有考虑 Buf_alloc 缓冲是否会出现上溢出（Overrun）或者负载（Underrun）。本节将首先给出 Overrun 和 Underrun 的定义，然后讨论 Credit-Based 机制使用的各类算法以及这些算法中使用的各类参数。

- 在本章中，Overrun 指 Current 节点没有足够的缓存接收来自 Upstream 节点的数据报

① 在网络系统中，Current 节点可能对应多个 Downstream 节点。

文；或者 Downstream 节点没有足够的缓存接收来自 Current 节点的数据报文。

- 而 Underrun 指 Downstream 节点有足够的缓存可以接收 Current 节点的报文，但是 Current 节点的缓存中没有需要发送的报文；或者 Current 节点有足够的缓存可以接收 Upstream 节点的报文，但是在 Upstream 节点的缓存中没有需要发送的报文。

这两种溢出情况都将导致链路带宽的浪费，在实际设计中需要尽力避免这两种溢出。此外在一个设计中，还需要考虑链路的传送延时。由于传送延时的存在，Current 节点向 Upstream 节点传送 Credit 信息时，这个 Credit 信息并不能瞬间到达，因而会造成这两个节点间，Credit 的同步问题。如果一个设计将上述这些因素考虑进去，Credit-Based 机制的实现更为复杂。为深入研究 Credit-Based 机制所使用的算法，我们首先定义以下系统参数。

(1) R_{TT} (Round Trip Time)

该参数记载数据通路的链路延时，单位为 s (秒)。使用 Credit-Based 机制进行报文传递时，Upstream 需要获得 Credit 然后才能发送报文。在链路中存在两个延时，一个是 Current 节点向 Upstream 节点发送 Credit 报文的线路延时 T_{CU} ，另一个是 Upstream 节点向 Current 节点发送数据报文的延时 T_{UC} 。

如果一个物理链路的发送与接收链路速度相等，而且 Credit 报文长度等于数据报文长度时， T_{CU} 将等于 T_{UC} 。但是在很多情况下这两个值并不相等。本章为简化起见，假设 T_{CU} 与 T_{UC} 的值相等。

而 R_{TT} 是这两种延时之和， R_{TT} 的值和物理链路的延时成正比。除此之外节点在发送数据报文时需要通过若干逻辑门，这也增加了传送延时。 R_{TT} 的值可以在链路配置时计算出来，但是在具体实现中，设计者可能使用一个事先预估的数值作为 R_{TT} 。值得注意的是，该参数不能估计得过低，否则将会造成 Current 节点的 Overrun；也不能估计得过高，否则将可能引发 Current 节点的 Underrun。

(2) BLINK

该参数存放 Upstream、Current 和 Downstream 节点间数据传递的峰值带宽，即数据链路所能提供的最大物理带宽，单位为 bit/s (Bits Per Second)。为简便起见，本章认为这几个节点间进行数据传递时的峰值带宽相等。

(3) Packet_Size

该参数存放一个数据报文的大小，单位为 bit。假定所有节点间进行数据通信时使用的数据报文的大小相等。值得注意的是，在 PCIe 总线中，数据报文并不等长，这为 PCIe 总线的流量控制带来了不小的麻烦。

(4) F (In-flight Data)

该参数存放在 R_{TT} 时间段内，Upstream 节点和 Current 节点之间的双向链路上存在的报文，其单位为报文数。其最大值等于 $R_{TT} \times B_{LINK} / \text{Packet_Size}$ 。该值在链路进行远距离传送时必须要考虑。而 PCIe 链路通常在一个 PCB 内部，至多作为机箱到机箱之间的链路，因此 RTT 的值非常小，F 参数几乎可以忽略不计。

(5) B_{VC}

该参数存放源节点到目标节点的有效数据带宽，单位为 bps。在一个物理链路上除了要传递有效数据之外，还需要传递 Credit 报文，因此 $B_{VC} < B_{LINK}$ 。

该参数不一定是一个常数，因为在一个实际的系统中，不同时间内的带宽并不一定相

等。为了简化模型，使用 B_{VCR} 参数替代 B_{VC} 参数， B_{VCR} 参数为一个常数。

本节力求简化流量控制的数学模型，并依此进行量化分析。有关流量控制的量化分析涉及一些相对复杂的数学推导，本章对此不做详细说明。

使用 Credit-Based 机制时，Buf_Alloc 参数可以被分解为三部分，分别由 N1、N2 和 N3 组成，Buf_Alloc 参数与 N1~N3 间的关系如图 9-6 所示。

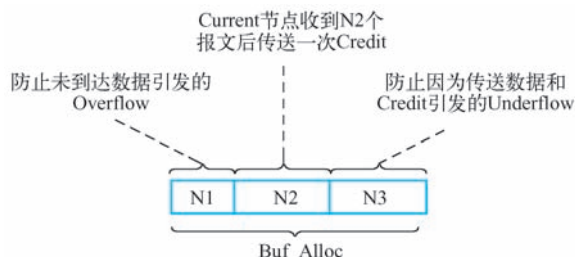


图 9-6 Buf_Alloc 的组成

(1) N1 缓冲

该参数用来防止因为线路延时而造成的 Overrun，Credit-Based 流量控制机制所采用的算法对该参数的解释有略微区别，详见下文。

(2) N2 缓冲

N2 的值与 VC 的设置有关。当 Current 节点向 Downstream 节点每转发 N2 个报文后，将向 Upstream 节点发送一个 Credit 报文。Current 节点可以将所有 VC 的 N2 值设为相同，也可以分别设置。

N2 的值越大，Buf_Alloc 参数的值也越大，节点发送 Credit 报文的频率也越低，从而 Credit 报文占用数据通路带宽的比例越小；N2 的值越小，则 Buf_Alloc 的值也越小，向 Upstream 节点发送的 Credit 报文的频率越高，Credit 报文占用数据通路带宽的比例也越大。如果 N2 的值为 10，Current 节点每转发 10 个报文后，才向 Downstream 发送一个 Credit 报文，从而整个系统用于传送 Credit 报文所占用的带宽不会超过 10%。

(3) N3 缓冲

该参数保证 Current 节点不会出现 Underrun，即出现 Downstream 节点仍有缓存接收报文，而在 Current 节点的缓存中没有报文需要发送这种情况。

为此在进行系统设计时，需要合理设置 N3 的值。使得在理想情况下，只要 Upstream 节点向 Current 节点不断地发送报文，且 Downstream 节点有足够的缓存时，就不会在 Current 节点中出现 Underrun 的现象。

当然 Downstream 节点接收数据报文的速度足够快，而且 Current 节点未能及时地从 Upstream 节点获得报文时，Current 节点总会出现 Underrun 的现象。

假设 Downstream 节点从 Current 节点获取报文的速度为 B_{VCR} ，只要 $N3 = B_{VCR} \times R_{TT} / \text{Packet_Size}$ ，那么在 Downstream 节点将 N3 中的报文取完之前，Current 节点总能获得新的报文（其前提是 Upstream 节点在不断地向 Current 节点发送数据报文）。

由此可见 B_{VCR} 与 N3 的容量有关， B_{VCR} 越大，N3 也越大。N3 的容量可以影响数据通路的有效带宽，在流量控制机制的实现中，如果 N3 过小，那么 Current 节点将出现

Underrun 的现象, 从而影响数据通路的有效带宽。根据上述数学模型, H. T. Kuang 与 Alan Chap-man 提出了三种流量控制算法, 分别为 N123、N123+和 N23 算法。

9.2.1 N123 算法和 N123+算法

N123 算法的使用规则如下所示。

1) Current 节点发送上一个 Credit 报文之后, 至少需要向 Downstream 节点转发 N2 个报文后, 才能发送下一个 Credit 报文。

2) Credit 报文中存放的 Credit 为 Current 节点已经发送的报文数, 其最大值为 $N2+N3$, 其中 N1 不参与“Credit”的计算, 因为在 N123 算法中, N1 用来防止 Current 节点的溢出。当 Current 节点使用的缓冲超过 $N2+N3$ 时, Current 节点不再向 Upstream 节点发送 Credit 报文, 即便 Current 节点已经向 Downstream 转发了 N2 个报文。

3) Upstream 节点收到 Credit 报文后, 使用 Crd_Bal 参数保存这个报文中的 Credit, 当 Crd_Bal 参数不为 0 时, Upstream 节点可以发送数据报文。Upstream 节点每发送一个报文, Crd_Bal 参数将减 1, 当这个参数为 0 时, Upstream 节点停止发送报文。Crd_Bal 参数的初始值为 $N2+N3$ 。

采用以上算法时, 必须保证 $N1 \leq R_{TT} \times B_{LINK} / \text{Packet_Size}$, 此时 Current 节点的接收缓冲才不会溢出。下文将详细解释为什么 N1 的最小值为 $R_{TT} \times B_{LINK} / \text{Packet_Size}$, 并用一个实例说明 N1 最小值的设置原因, 而不进行理论分析。

假设在某一个时间点, Upstream 节点的 Crd_Bal 参数为 0, 而 Current 节点的 N2 和 N3 缓冲区被完全占满, 此时 Upstream 节点不能发送新的报文, 直到获得新的 Credit 报文。之后 Upstream、Current 节点和 Downstream 节点按照以下步骤运行。

1) 当 Current 节点向 Downstream 节点转发 $x (x \geq N2)$ 个报文后, 将通过 Credit 报文将 x 传递给 Upstream 节点。

2) 当 Upstream 节点收到这个 Credit 后, 将更新 Crd_Bal 参数为 x , 之后可以向 Current 节点发送 x 个报文。这些报文将通过链路不断发向 Current 节点。

3) 假设此时 Current 节点接收到的报文个数为 $z1 (z1 \leq x)$, Current 节点在接收这些报文的同时, 向 Downstream 节点又转发了 y 个报文, 此时 Current 节点一共需要向 Upstream 节点发送的 Credit 为 $(x+y-z1)$ 。假定此时 Downstream 节点由于缓存不足, 禁止接收来自 Current 节点的报文, Current 节点的空闲缓存将定格为 $x+y-z1$ 。

4) 在 Current 节点向 Upstream 节点发送 Credit (该值为 $x+y-z1$) 的过程中, 该节点又陆续收到了一些报文, 其个数为 $z2$ 。值得注意的是“Current 节点发送 Credit 报文”到“Upstream 节点收到这个报文”有一段延时, 该延时等于 T_{CU} 。

5) Upstream 节点收到新的 Credit 后, 将 $x+y-z1$ 个报文发向 Current 节点, 除此之外在从 Upstream 节点到 Current 节点间的链路上还留有一部分报文, 其个数为 $z3$ 。这段残留的报文数为在 T_{UC} 时间段内传递的数据报文。

6) 因此 Current 节点最终收到的报文个数为 $(x+y-z1) + z2 + z3$ 个。

7) 其中 $x+y-z1$ 个报文可以被 Current 节点中空闲缓存吸收, 而多出来的 $z2+z3$ 个数据报文将放置到 N1 中。

$\text{Max}(z2+z3)$ 的计算方法如式 9-1 所示。

$$\begin{aligned}\text{Max}(z2+z3) &= (T_{\text{CU}} \times B_{\text{LINK}} + T_{\text{UC}} \times B_{\text{LINK}}) / \text{Packet_Size} \\ &= R_{\text{TT}} \times B_{\text{LINK}} / \text{Packet_Size}\end{aligned}\quad (9-1)$$

因此只要 N1 被设置为 $R_{\text{TT}} \times B_{\text{LINK}} / \text{Packet_Size}$ ，采用 N123 算法就一定不会在 Current 节点上产生 Overrun。

通过以上分析，可以发现由于物理链路传送的延时，采用 N123 算法时，Current 节点将多收到 $z2+z3$ 个报文，其中 $z2$ 是 Current 节点更新 Upstream 节点 Credit 的过程中产生；而 $z3$ 是 Upstream 节点更新完毕 Credit 后，在网络线路上残留的报文。使用 N123 算法时，需要设置 N1 缓冲处理这些因为网络延时产生的报文。

以上过程并非严格的数学证明，只是使用较为简单的推理说明，在某些情况下，在 Current 节点中的 N1 至少为 $R_{\text{TT}} \times B_{\text{LINK}} / \text{Packet_Size}$ 时，才能够保证 Current 节点的接收缓冲不会溢出。希望读者认真理解 N1 缓冲在 N123 算法中的意义。

N123+算法基于 N123 算法，但是发送 Credit 报文的方式略有不同。N123 算法要求 Current 节点每转发 N2 个报文后，才能给 Upstream 节点发送一个 Credit 报文；而 N123+算法除了要求同样的规则之外，还要求 Current 节点在一个时间段 R_{TT} 中至少发送一个 Credit 报文，即发送上一个 Credit 报文之后，即便 Current 节点没有向 Downstream 转发了 N2 报文，在一个时间段 R_{TT} 中也至少要向 Upstream 节点发送一次 Credit 报文。

采用这种方法，因为在每一个 R_{TT} 时间段里都会向 Upstream 节点发送 Credit 信息，因此 F 将小于这个 Credit，而这个 Credit 又小于 $N2+N3$ 。为此使用这种方法时，N1 的计算方法如式 9-2 所示。

$$N1 = \text{Min}(N2+N3, R_{\text{TT}} \times B_{\text{LINK}} / \text{Packet_Size}) \quad (9-2)$$

使用这种方法，在 $(N2+N3) < R_{\text{TT}} \times B_{\text{LINK}} / \text{Packet_Size}$ 时，Current 节点将可以使用较小的 N1 缓冲，从而节约了接收缓冲。这种算法对于网络延时较长的通信网络有所帮助，而网络延时较短时，但是采用这种方法，发送 Credit 报文所占用的带宽较大。在 PCIe 总线中，端到端的延时相对较小，因此没有必要使用这种流量控制机制。

9.2.2 N23 算法

N23 算法是流量控制中常用的算法，使用该算法的优点是 Current 节点的缓存中不包含 N1，从而降低了节点的缓存容量。该算法基于 N123 算法，区别在于使用该算法时 Crd_Bal 参数的计算。基于该算法的实现方式如图 9-7 所示。

N23 算法的使用规则如下。

- 当系统初始化时，Crd_Bal 参数为 $N2+N3-E$ ，E 为在时间段 R_{TT} 中，Upstream 节点向 Current 节点发送的报文数，其值等于 $B_{\text{VCR}} \times R_{\text{TT}} / \text{Packet_Size}$ 。而 Upstream 节点每发送一个报文，Crd_Bal 参数的值将减 1。当 Crd_Bal 参数等于 0 时，Upstream 节点停止发送报文，直到重新获得 Current 节点的 Credit 报文，更新 Crd_Bal 参数后，才能继续发送。
- 使用 N23 算法时，Crd_Bal 参数与 Current 节点发出的 Credit 并不相等，而是等于 $\text{Credit}-E$ 。
- Current 节点至少需要向 Downstream 节点发送 N2 个报文后，才能向 Upstream 节点发送 Credit 报文，与 N123 算法一致。

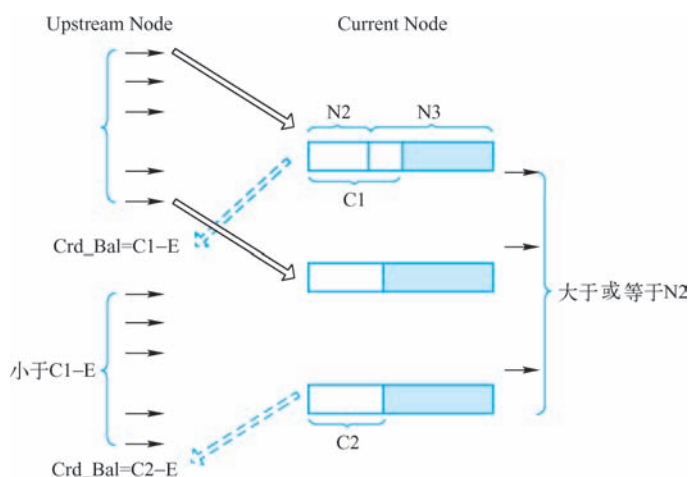


图 9-7 基于 N23 算法的流量控制

通过以上介绍，可以发现之所以采用 N23 算法，不需要设置 N1，是因为 Upstream 节点使用的 Crd_Bal 参数与 N123 算法相比少 E 个包，所以 N23 算法虽然没有使用 N1 缓冲，也不会导致在传送过程中出现 Overrun，因为采用 N23 算法，将 N1 隐含在 E 中。同时因为 N3 的存在，采用 N23 算法也不会导致在传送过程中出现 Underrun。

综上所述，使用 N23 算法进行数据报文的传递时，只要 N2 和 N3 参数设置合理，将不会发生节点的 Overrun 和 Underrun 的情况。但是还需要继续讨论 Current 节点发送 Credit 报文时会不会引发 Overrun 和 Underrun。

首先 Current 节点发送 Credit 报文不会引发 Upstream 节点的 Overrun，因为 Upstream 节点每次接收到新的 Credit 报文都会把 Crd_Bal 参数更新，不可能因为 Credit 报文过多而无法处理。但是 Current 节点发送过多的 Credit 报文将严重影响物理链路的有效利用率，在流量机制的实现中，需要合理设置发送 Credit 报文的频率。

而 Current 节点向 Upstream 节点传递 Credit 报文延时过大时，可能会引发 Current 节点的 Underrun。在这种情况下，Upstream 节点虽然有很多报文等待发送，但是由于 Crd_Bal 参数为 0，不能发送这些报文。

因此造成在 Current 节点的数据缓冲中，没有数据报文需要发向 Downstream 节点，尽管此时在 Current 节点中还有足够的缓存可以接收数据报文。在流量控制机制的设计中，需要考虑 Credit 报文的传送延时，合理设置 Current 节点中的缓冲，以保证 Upstream 节点在获得新的 Credit 之前，Current 节点的缓冲中具有一定的报文数，以避免 Underrun。

采用 N23 算法可以有效避免这种因为 Credit 报文传递不及时而引发的 Underrun（N123 算法与 N23 算法都使用了 N3 缓冲避免 Current 节点的 Underrun）。我们首先基于 N23 算法做出以下假设以简化数学模型。

1) $N3 = B_{VCR} \times R_{TT} / \text{Packet_Size}$ 。设置 N3 缓存的主要目的是保证 Current 节点不会出现 Underrun。而 $B_{VCR} \times R_{TT} / \text{Packet_Size}$ 是 N3 缓存的最小值。

2) Upstream、Current 和 Downstream 间 VC 的通信带宽为 B_{VCR} ，其值为一个常数。

3) Downstream 节点始终有足够的缓冲接收报文。Current 节点可以通畅地将数据报文转

发到 Downstream 节点。

Upstream 和 Current 节点间的数据交换是基于“得到 Credit，然后发送数据”这样的循环。假定在一个发送循环的起始点中，Upstream 节点的 Crd_Bal 参数为 $N2$ ，即 Upstream 节点刚收到的 Credit 为 $N2+N3$ ，而采用 N23 算法时， $Crd_Bal = Credit - E = (N2+N3) - E = N2$ 。

我们定格 Upstream 节点刚刚收到 Current 节点 Credit 报文，更新 Crd_Bal 参数完毕这个场景，其时间戳为 $T2$ ，而 Current 节点发送这个 Credit 报文的时间戳为 $T1$ 。假设从 $T1 \sim T2$ 这段时间内，Current 节点收到 $z2$ 个报文（Current 节点向 Upstream 节点发送 Credit 报文的过程中，仍然在持续地接收报文）。 $T1 \sim T3$ 的示意如图 9-8 所示。

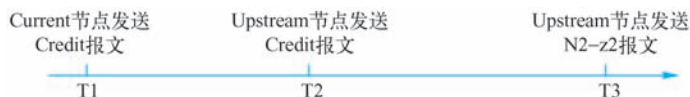


图 9-8 $T1 \sim T3$ 的示意图

由于各节点的带宽都为 BVCR，所以 Current 节点每收到一个报文，都会发送到 Downstream 节点，因此此时 Current 节点的可用缓存始终保持为 $N2+N3$ 。Current 节点向 Upstream 节点发送的 Credit 报文始终为 $N2+N3$ ，而 Upstream 节点收到这个 Credit 后，其 Crd_Bal 参数将被置为 $N2$ 。

Upstream 节点收到 Credit 报文后，开始向 Current 节点发送 $N2$ 个报文，Upstream 节点每发送一个报文，Current 节点将收到一个报文（假设此时从 Upstream 节点到 Current 节点的链路之间已经堆积了 $z2$ 个报文，此时物理链路已经充满数据报文）。

假设 Upstream 节点向 Current 节点发送完毕 $N2-z2$ 个报文（数据报文离开发送端口的时间戳为 $T3$ ）。那么从 $T3 \sim T1$ 这段时间里，Current 节点将收到 $N2$ 个报文，同时 Current 节点也将向 Downstream 节点转发完毕 $N2$ 个报文，此时（ $T3$ 时间戳）Current 节点将向 Upstream 节点发送 Credit 报文（数值为 $N2+N3$ ）。

而 Upstream 节点处于 $T3$ 这个时刻时，还能向 Current 节点发送 $z2$ 个报文，因为之前已经发送了 $N2-z2$ 个报文，此时 Crd_Bal 参数为 $z2$ 。等到 Upstream 节点将 $z2$ 个报文发送完毕，来自 Current 节点的 Credit 报文恰好到达，因为 Upstream 节点将 $z2$ 个报文发送完毕的时间刚好等于 Current 节点向 Upstream 节点发送 Credit 报文的时间。通过以上计算，可以发现采用 N23 算法不会因为 Credit 报文的传送时间而导致 Current 节点的 Underrun。

此外采用 N23 算法时还需要处理错误报文，N23 算法规定当一个节点收到一个错误数据报文时，将丢弃这个报文，此时这个被丢弃的报文将不占用接收节点的缓存，但是发送节点的 Crd_Bal 参数仍然需要考虑这个报文。

9.2.3 流量控制机制的缓冲管理

上文讲述了基于单个 VC 的流控机制，实际上在 Upstream、Current 和 Downstream 节点中一般含有多个 VC。多个 VC 之间如何合理地使用缓存值得重点关注，在实际设计中，可以为每一个 VC 设置独立接收缓存，也可以使多个 VC 共享同一个接收缓存。在 FCVC 的实现中，可以根据实际情况选择独立缓存或者共享缓存。

其中，每一个 VC 都使用独立接收缓存的流量控制方法称为静态（Static）流量控制；

而使用共享缓存的流量控制方法称为自适应（Adaptive）流量控制。

假定在一个系统中，一共具有 n 条 VC，而且这几条 VC 都使用 N23 算法进行流量控制，那么在使用 Static 流量控制方式时，该系统一共需要的缓冲大小为 $(n \times N2 + N3) \times \text{Packet_Size}^{\ominus}$ 。如果 $(n \times N2 + N3) \times \text{Packet_Size}$ 并不是很大时，为了使数据链路获得更大的带宽，可以使用 Static 流量控制。使用这种方法，也将极大地简化缓冲管理的设计难度。

值得注意的是，在一个系统工程的架构设计中，应当重点关注“Critical Path”的设计，需要容忍非“Critical Path”的不完美。当 $(n \times N2 + N3) \times \text{Packet_Size}$ 的值大到了可以容忍的范围之外时，设计者必须考虑如何减少 Current 节点的接收缓存大小。Static 流量控制是针对每一个 VC 都是按照全负荷运转的情况，在绝大多数应用中，几乎不可能出现每一条 VC 都被充分利用的情况，因为多条 VC 共享一个物理链路，不可能出现所有 VC 都在全负荷运行的情况。为此在系统设计时可以使用 Adaptive 流量控制方法。

Adaptive 流量控制的本质是 Current 节点中，所有 VC 共享一个接收缓存，从而这个缓存可以远小于 $(n \times N2 + N3) \times \text{Packet_Size}$ 。因为在绝大多数时间内，数据链路的多条 VC 不可能都被充分使用，因此并不需要为每条 VC 都提供 N2 缓冲，而是为所有 VC 统一提供接收缓冲，从而合理使用这些接收缓冲。

目前接收缓存的分配常使用两种算法，分别是 Sender-Oriented 和 Receiver-Oriented 管理算法。这两种算法的缓冲设置如图 9-9 所示。使用 Sender-Oriented 管理算法时，Adaptive Buffer 的分配在 Upstream 节点中完成，如果系统中有多个 Upstream 节点，Current 节点需要在其接收端点处为每个 Upstream 节点准备 Adaptive Buffer，而且 Current 节点并不知道 Upstream 节点的使用情况，这为 Current 节点的缓冲管理带来了不小的困难。

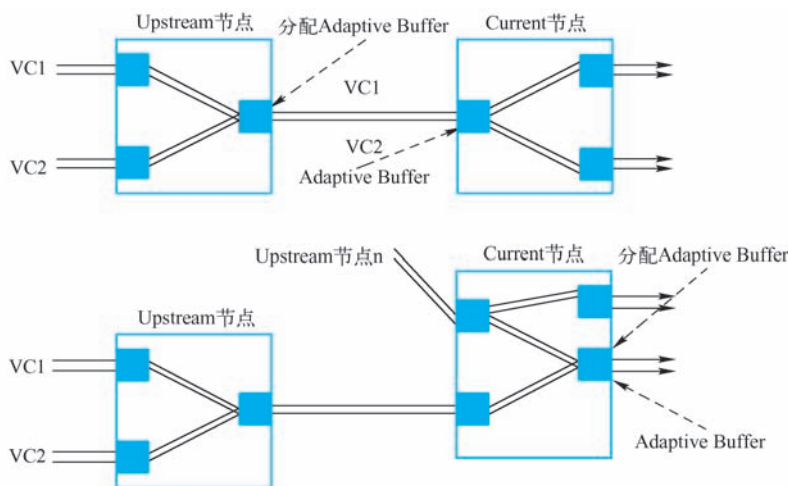


图 9-9 Sender-Oriented 和 Receiver-Oriented 管理算法

而使用 Receiver-Oriented 管理算法可以有效避免这类困难，使用这种算法时，所有来自 Upstream 节点的数据报文在 Current 节点的发送端准备一个 Adaptive Buffer，对这个 Buffer 的分

[⊖] 多条 VC 可以复用 N3 缓冲，因为多条 VC 对应一条物理链路。但是必须提供独立的 N2 缓冲。

配也在 Current 节点内部完成。这两种算法的具体实现都较为复杂，本节并不详细介绍这些算法，在 PCIe 总线中，RC 和 Switch 的硬件设计将会涉及这些内容，而 EP 无需关心这些问题。

图中的上半部分是 Sender-Oriented 管理算法的示意图，而下半部分是 Receiver-Oriented 管理算法的示意图。由图 9-9 可以发现，使用这两种算法时 Adaptive Buffer 都在 Current 节点中，只是位置不同。

9.3 PCIe 总线的流量控制

我们仍然使用上文的 Upstream、Current 和 Downstream 节点模型分析 PCIe 总线使用的流量控制机制。在 PCIe 总线中，Upstream 节点和 Downstream 节点可以为 RC 或者 EP，而 Current 节点只能为 Switch 或 RC[⊖]。

此外在 PCIe 总线中，允许 Upstream 节点和 Downstream 节点直接相连，而不需要经过 Current 节点，如 RC 的某个端口可以和 EP 直接相连。当然这种情况也可以理解为 Upstream 和 Current 节点直接相连，但是 Current 节点不需要与 Downstream 节点相连。

与传统的流量控制机制相比，PCIe 总线的流量控制机制有所不同。流量控制机制首先出现在互联网中，使用流量控制机制最典型的应用是基于 ATM（Asynchronous Transfer Mode）的分组交换网。在 ATM 分组交换网系统中，数据传递以报文为单位进行，每一个报文都可以独立地通过分组交换网，到达目的地。而 PCIe 总线的数据传递基于节点到节点间的数据传递，一个完整的 PCIe 总线传输需要使用多个报文，而且这些报文和报文之间还有一定的联系，如一个完整的存储器读由存储器读请求 TLP 和存储器读完成 TLP 组成。

在 PCIe 总线中，RC 端口和 EP 之间可以直接互连，而不需要中间节点，这和基于分组交换的网络有很大的不同。此外在 PCIe 总线中，报文的大小并不固定，如数据报文的大小可以为 128B、256B，只要数据报文的有效负载小于 Max_Payload_Size 参数即可。

这些长度不确定的数据报文，为 PCIe 总线的流量控制带来了不小的困难，也是因为这个原因，PCIe 总线的流量控制机制将一个 TLP 分为 TLP 头和 Payload 两部分，并分别为这两部分提供不同的接收缓冲，以合理利用 PCIe 链路的带宽。

PCIe 总线的这些特性实际上不利于流量控制的实现，为此 PCIe 总线在传统流量控制理论的基础上，做出了许多改动。在 PCIe 总线中存在多条 VC，其流量控制也是基于 FCVC 的，但是 PCIe 总线在接收缓冲的设计上与传统的流量控制机制有很大的不同。

PCIe 总线的主要应用领域在 PC 或者服务器中进行板内互连，在这个应用领域中，流量控制并不是最重要的。PCIe 总线的流控机制远非完美，这在某种程度上影响了 PCIe 总线在大规模互连结构中的使用。但是 PCIe 总线的流量控制机制仍有其闪光之处，因此读者仍有必要了解 PCIe 总线的流量控制机制。

与传统流量控制机制相比，PCIe 总线在实现流量控制机制时需要更多的接收缓存，因此 PCIe 总线实现流控的代价相对较大。值得庆幸的是，目前 PCIe 总线上的设备，包括 RC、EP 和 Switch，很少有支持两个以上 VC 通路的。一般来说 PCIe 总线上的设备，如 RC 和

[⊖] 如果 RC 支持 Peer-to-Peer 传送方式。

Switch 上也只有两个 VC，多数 EP 仅支持一个 VC。

PCIe 总线的流量控制机制由事务层和数据链路层协调实现，而对系统软件透明。PCIe 总线使用 Credit-Based 流量控制机制，其中 Credit 报文以 DLLP 的形式从 Current 节点反馈到 Upstream 节点。在 PCIe 总线中，数据报文首先以 TLP 的形式通过数据链路层，而到达数据缓存时被分解为 Header 和 Data 两个部分，分别存放到不同的接收缓存队列中。

9.3.1 PCIe 总线流量控制的缓存管理

在 PCIe 总线的节点中，一个 VC 的接收缓存由 PH（Posted Header）缓存、PD（Posted Data）缓存、NPH（Non-Posted Header）缓存、NPD（Non-Posted Data）缓存、CplH（Completion Header）缓存和 CplD（Completion Data）缓存组成。

- PH 缓存存放存储器写请求 TLP 和 Message 报文使用的 TLP 头。
- PD 缓存存放存储器写请求 TLP 和 Message 报文使用的 Payload。
- NPH 缓存存放 Non-Posted 请求 TLP 使用的 TLP 头。
- NPD 缓存存放 Non-Posted 请求 TLP 使用的 Payload。在 Non-Posted 请求 TLP 中，如存储器读请求 TLP 并不含有 Payload 字段，但是 I/O 和配置写请求 TLP 使用 Payload 字段。
- CplH 缓存存放完成报文使用的 TLP 头。
- CplD 缓存存放完成报文使用的 Payload。如上文所述，完成报文分为两大类，带数据的完成报文和不带数据的完成报文。其中不带数据的完成报文不需要使用 CplD 缓存。

在 PCIe 总线中，一个 TLP 从 Upstream 节点传送到 Current 节点时，必须同时具备多个缓存的 Credit 后才能发送。如存储器写请求 TLP，需要同时具备 PH 和 PD 缓存的 Credit，才能发送；而“不带数据的”存储器读完成 TLP，仅需要具备 CplH 缓存即可。这些缓存在 PCIe 设备中的组成结构如图 9-10 所示。

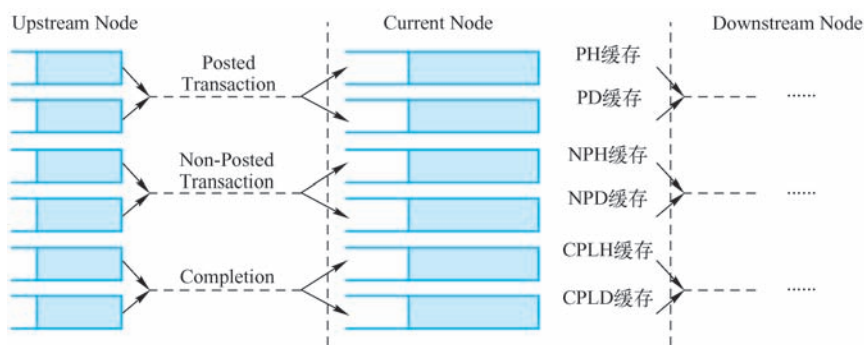


图 9-10 PCIe 总线 Current 节点的缓存管理

如表 6-2 所示，PCIe 总线根据 Type 字段将 TLP 分为 22 种，而根据这些 TLP 的传输特性，可以将这些 TLP 分为 Posted Transaction、Non-Posted Transaction 和 Completion 三大类。在 PCIe 总线中，这三大类数据传送需要遵循各自的规则，这些 Transaction 也有各自的特点。这三大类 Transaction 在进行数据传递时需要使用不同的缓存，这些缓存由多个单元（Unit）组成。每个 Unit 的大小与缓存类型相关，如表 9-1 所示。

表 9-1 PCIe 总线缓存的单元大小

类 型	Unit 大小
PH 缓存	5 个 DW，Posted Transaction TLP 头的最大值为 4DW，再加上一个 Digest
PD 缓存	4 个 DW
NPH 缓存	5 个 DW，Non-Posted Transaction 的 TLP 头的最大值为 4DW，再加上一个 Digest
NPD 缓存	1 个 DW。NPD 缓存用于 IO 和配置读写的数据
CplH 缓存	4 个 DW。3DW 的完成报文的头，加上一个 Digest
CplD 缓存	4 个 DW。该缓存至少能够存放一个完整的带数据的完成报文。其总单元数的最小值为 $\min(\text{RCB}/4\text{B}, \text{Max_Payload_Size}/4\text{B})$

PCIe 总线将 Header 和 Data 缓存分离的主要原因是，一个 TLP 的 Header 大小是固定的，如 PH 和 NPH 大小在 5DW 之内，CplH 大小在 4DW 之内；而 Data 的大小并不固定，除了 NPD 的大小为 1 个 DW 之外，其他数据报文的长度由 TLP 的 Length 字段确定，并不固定。PCIe 总线将 Header 和 Data 缓存分离有利于 Data 缓存的合理使用。

PCIe 总线规范将这些缓存使用的 Unit 统称为 FC（Flow Control）Unit，下文将以 FC Unit 简称这些对应缓存的 Unit。因为 Header 的大小固定，所以 Header 缓存能够精确地预知可以容纳几个 Header；而由于 Data 的大小并不固定，Data 缓存无法精确预知可以存放几个 Data，当然 Data 缓存也不可能将 Data 的基本单位设置为 Max_Payload_Size 参数。因为这样做不仅不合理，而且非常浪费资源。将 Header 和 Data 缓存分离，有利于 Data 缓存使用类似 Adaptive 流量控制的方法使所有 Data 共用一个缓存，从而提高了 Data 缓存的利用率。但是也造成 TLP 因为不能同时具有 Head 和 Data 缓存，而无法传送。

在 PCIe 总线中，不同的 TLP 使用对应缓存的 Unit 数量也不同，Current 节点有时需要两种缓存才能接收一个 TLP。不同 TLP 使用的缓存数目如表 9-2 所示。

表 9-2 不同 TLP 使用的缓存

TLP	缓存种类
存储器、IO 和配置读请求	使用 1 个 NPH 单元。Non-Posted 数据请求的 TLP 类型通过完成报文获得数据
存储器写请求	使用 1 个 PH 单元和若干 PD 单元。存储器写请求使用的 PD 单元与 Payload 的长度相关
配置、I/O 写请求	使用 1 个 NPH 单元和 1 个 NPD 单元。Non-Posted 数据传送类型通过完成报文通知 I/O 写结束
不带数据的消息请求	使用 1 个 PH 单元
带数据的消息请求	使用 1 个 PH 单元和若干个 PD 单元
存储器读完成	使用 1 个 CplH 单元和若干 CplD 单元。读完成报文使用的 CplD 单元与 Payload 的长度相关
I/O 和配置读完成	使用 1 个 CplH 单元和 1 个 CplD 单元
配置、I/O 写完成	使用 1 个 CplH 单元

由上表所示，一个 TLP 可能需要使用两种缓存，如存储器写请求需要使用 PH 和 PD 缓存。这也意味着在 PCIe 设备的 VC 中，缓存之间存在依赖关系。当 Upstream 节点向 Current 节点进行存储器写时，发送方必须同时具有 PH 和 PD 两个缓存的 Credit 才能进行；而向 Current 节点发送读完成 TLP 时，Upstream 节点必须同时具有 CplH 和 CplD 两个缓存的 Credit 才能进行。这为 PCIe 总线的流量控制带来了额外的麻烦。

此外，在 PCIe 总线中，进行存储器写和存储器读完成 TLP 时，究竟需要多少个 PD 或者 CplD 单元是随 TLP 而变的，VC 无法预知确切的单元数量。无论 VC 采用何种缓冲分配策

略，这种“不可预知”都会给流量控制带来巨大的麻烦。

报文长度的不确定性为 PCIe 总线流量控制机制带来了许多难以解决的问题。造成这种现象的主要原因是 PCIe 总线源于 PCI 总线，其主要应用来自 PC 领域而不是通信领域。PCIe 总线为了向前兼容 PCI 总线，做出了许多功能上的牺牲。

PCIe 总线使用 Credit-Based 流控机制，Upstream 节点在发送 TLP 时，必须首先获得 Current 节点相应缓存的 Credit。如 Upstream 节点发送存储器写请求时，需要同时具有 Current 节点中 PH 缓存和 PD 缓存的 Credit，才能进行。

9.3.2 Current 节点的 Credit

PCIe 总线规范没有强行规定采用哪种算法实现 Credit-Based 流量控制机制，因此 PCIe 总线上的 Current 节点可以选择使用合理的 Credit-Based 流量控制算法，如上文提到的 N123 算法或者 N123+算法。PCIe 总线规范没有规定，各类节点如何处理因为链路延时而产生的额外报文和在链路上残留的报文。

在 PCIe 流量管理中还存在一个问题就是 PCIe 总线上各个节点所采用的流量控制算法并不完全统一，虽然 PCIe 总线对此进行了一定程度的约定，但是这个约定较为模糊。因为在 PCIe 总线中，Upstream 节点、Current 节点和 Downstream 节点可能来自不同的生产厂商。虽然这种流量控制算法的“不统一”会为流量控制的整体效率带来影响，也可能造成接收缓冲的浪费。

但是这种“不统一”并不会严重影响 PCIe 总线的流量控制机制。因为 PCIe 总线的流量控制基于“链路到链路”进行的，只要链路的两端采用相同的协议满足 PCIe 总线的基本约定即可。PCIe 总线上的流量控制机制以 Intel 的 RC 实现作为事实标准。

在 PCIe 总线中，Credit-Based 流量控制的数据传送规则仍然是 Upstream 节点获得 Credit，之后向 Current 节点发送数据；而 Current 节点将一定数量的报文（N2）转发到 Downstream 节点之后，将向 Upstream 节点反馈 Credit。PCIe 总线规范也将向 Upstream 节点反馈 Credit 的过程称为 Advertisement。

PCIe 总线规范并没有规定如何设置 Credit，但是规定了 Credit 在初始化之后的最小值，即 Current 节点发向 Upstream 节点的 Credit 的最小值，如表 9-3 所示。PCIe 总线中，不同的缓存使用不同的 Credit 值。

表 9-3 PCIe 总线初始化后 Credit 的最小值

Credit 的类型	Credit 的最小值
PH 缓存	Credit (PH) 的最小值为 0x01，即一个 PH 单元，大小为 5DW
PD 缓存	Credit (PD) 的最小值为 Max_Payload_Size/Unit(PD)
NPH 缓存	Credit (NPH) 的最小值为 0x01，即一个 NPD 单元，大小为 5DW
NPD 缓存	Credit (NPD) 的最小值为 0x01，即一个 NPH 单元，大小为 1DW。值得注意的是原子操作请求使用的 Credit (NPD) 可以为 2DW
CplH 缓存	如果 Current 节点不是“最终”节点，则 Credit (CplH) 的最小值为 0x01，即一个 CplH 单元，大小为 4DW；否则 Credit (CplH) 为 0，该值为 0，表示“不限带宽 (Infinite FC Unit)”，其详细解释见下文
CplD 缓存	如果 Current 节点不是“最终”节点，则 Credit (CplD) 的最小值为 Max_Payload_Size/Unit(CplD) ^① ；否则 Credit (CplH) 为 0

① 注意不是 RCB/Unit (CplD)。第一个“最终”节点的 RCB 并不相等，但是都小于 Max_Payload_Size。

“最终节点”共有两类组成，一类是 EP，因为当一个报文到达 EP 后，将不会被再次转发，此时 FC Unit 为 0，表示该节点将无条件接收来自 Upstream 节点的报文，而且保证一定不会出现 Overrun 的现象，这就是 PCIe 总线规范提出的 Infinite FC Unit 的含义。

这也意味着 EP 在发起存储器、I/O 和配置读请求时，必须为存储器、I/O 和配置读完成报文预留必要的缓存，保证这些完成报文一定能够被 EP 接收。值得注意的是，只有设备在接收完成报文时，才存在 Infinite FC Unit 的概念。

在许多应用中，“最终节点”支持这种“Infinite FC Unit”将会影响 PCIe 链路的效率。假设一个 PCIe 设备进行 DMA 读，从主存储器获得数据，如果该 PCIe 设备支持 Infinite FC Unit，其传送流程如下所示。

- 1) PCIe 设备向 RC 连续发送存储器读请求 TLP。
- 2) PCIe 设备每发送一个读请求 TLP 时，将从接收缓冲中为读完成 TLP 预留空间。因为如果 PCIe 设备支持 Infinite FC Unit，必须能够接收这些读完成 TLP。
- 3) 当 PCIe 设备的接收缓冲使用完毕后，将不能发送存储器读请求 TLP。
- 4) PCIe 设备接收来自 RC 的读请求完成 TLP，并将其传送到上层，同时释放该读完成 TLP 使用的接收缓存。
- 5) PCIe 设备获得可用的接收缓存后，可以继续发送存储器读请求 TLP。直到完成所有数据请求。
- 6) PCIe 设备获得所有存储器读完成 TLP 后，结束整个 DMA 读过程。

由以上过程可以发现 3) 和 5) 中，由于接收缓冲不足，整个 DMA 读过程无法形成连续的流水操作，从而影响 DMA 读的数据传送效率。如果 PCIe 设备的接收缓冲无限大时，可以合理地解决这个问题，但是更为合理的方法是确定接收缓冲的最小值。接收缓冲的最小值由 PCIe 链路的延时决定，有关 PCIe 总线延时的详细分析见第 12.4 节。

还有一类“最终节点”是 RC，但 RC 并非在任何情况下都是最终节点。如果多端口 RC 的各个端口之间不支持转发（即 PCIe 总线规范中的 Peer-to-Peer 传送方式）时，RC 将成为“最终节点”；如果多端口 RC 的各个端口之间支持转发，RC 也可能成为“最终节点”。数据通过多端口 RC 的流程如图 9-11 所示。

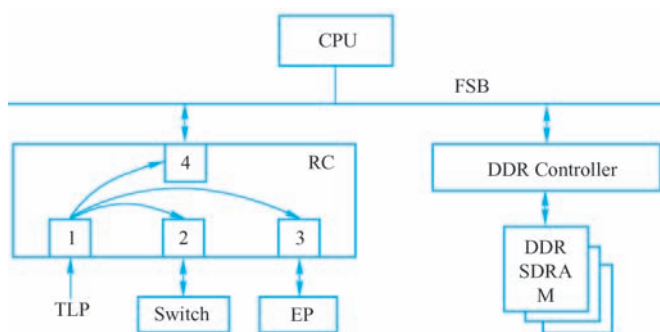


图 9-11 多端口 RC 的数据传递

假设在一个 RC 中有 3 个下游端口，分别是端口 1，2，3，这些端口与 EP 或者 Switch 相连；还有一个上游端口，端口 4，与 FSB 相连。如果 RC 支持端口转发，当到达端口 1 的

TLP 的目的地是和端口 2 或者 3 相连的某个设备时，Credit (CplH) 的最小值为 1，而 Credit (CplD) 的最小值为 Max_Payload_Size/Unit(CplD)，此时 RC 并不是最终节点。

如果到达端口 1 的 TLP，其目的地是端口 4 时，RC 就是“最终”节点，此时 Credit (CplH) 和 Credit (CplD) 的最小值为 0，流量控制机制不起作用，来自 Upstream 节点的报文可以不进行流量检测，直接进入 RC，而 RC 必须保证有足够的缓存接收这些报文。这也意味着 RC 发起读请求报文时，必须保证在 RC 中有足够的缓冲接收完成报文。

如果 RC 支持端口间的相互转发，必须设置必要的缓冲以支持流量控制机制，此时 RC 可能成为某个 TLP 的 Current 节点，因此 RC 不将 Credit (CplH) 和 Credit (CplD) 设为 0，此时使用流量控制机制进行报文转发。

9.3.3 VC 的初始化

PCIe 总线使用 FCP (Flow Control Packets) 传递 Credit 信息，FCP 是一种 DLLP，该报文的使用与事务层的接收缓存直接相关，但是对事务层透明，该报文产生于数据链路层，终止于数据链路层。PCIe 总线定义了以下 FCP，如表 9-4 所示。

表 9-4 PCIe 总线定义的 FCP

DLLP 类型	编 码
InitFC1-P	0b0100 0V ₂ V ₁ V ₀ ^①
InitFC1-NP	0b0101 0V ₂ V ₁ V ₀
InitFC1-Cpl	0b0110 0V ₂ V ₁ V ₀
InitFC2-P	0b1100 0V ₂ V ₁ V ₀
InitFC2-NP	0b1101 0V ₂ V ₁ V ₀
InitFC2-Cpl	0b1110 0V ₂ V ₁ V ₀
UpdateFC-P	0b1000 0V ₂ V ₁ V ₀
UpdateFC-NP	0b1001 0V ₂ V ₁ V ₀
UpdateFC-Cpl	0b1010 0V ₂ V ₁ V ₀

① V₂V₁V₀ 与 VC 对应，PCIe 总线规定 VC 个数的最大值为 8，因此使用 3 位存放 VC 号。

由上表所示 FCP 共分为三大类 InitFC1、InitFC2 和 UpdateFC。在这三类报文中 有 3 个重要的字段。其中 HdrFC 字段存放 Header 的 Credit；DataFC 字段存放 Data 的 Credit；而 VC ID 字段存放不同的 VC 号。其报文格式如图 9-12 所示。

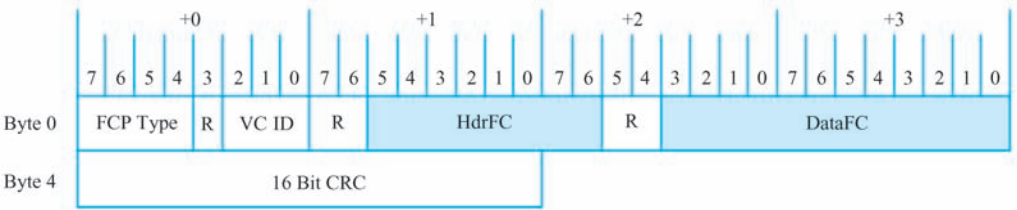


图 9-12 FCP 的格式

Current 节点使用以上报文向 Upstream 节点发送 Credit 信息，其中 InitFC1 和 InitFC2 与 VC 的初始化相关，而 UpdateFC 负责向 Upstream 节点反馈 Credit 信息。我们首先讲述 Current 节点如何使用 InitFC1 和 InitFC2 报文初始化 VC 的流量控制，并在第 9.3.4 节讲述如何使用 UpdateFC 报文实现 PCIe 总线的流量控制。

在各个节点能够正常使用之前，首先需要对 VC0 进行初始化。当 VC0 初始化完毕之后，PCIe 设备可以对 VC1~7 进行初始化。在 VC0 初始化完成之前，设备不能接收任何 TLP，而 VC0 初始化完毕后，PCIe 总线的相应节点在使用 VC0 接收 TLP 的同时，初始化其他 VC。在 VC 的初始化过程中存在两个状态，FC_INIT1 和 FC_INIT2。

PCIe 总线的数据链路层共有三个状态，分别为 DL_Inactive（PCIe 链路无效状态，链路不可用或者链路上没有连接有效设备）、DL_Init（PCIe 链路可用，此时将进行 VC0 的初始化）和 DL_Active（链路可以正常使用）。

当 Current 节点的数据链路层进入 DL_Init 状态时，该节点的 VC0 将进入 FC_INIT1 状态。Current 节点将在 FC_INIT1 状态首先初始化 VC0，之后才能初始化其他 VC。其他 VC 的使能位在 Current 节点的配置空间中，当系统软件打开这些使能位时，Current 节点将初始化其他 VC。

当 Current 节点的 VC 进入 FC_INIT1 状态时，事务层需要首先禁止该 VC 发送数据报文，随后 Current 节点将向 Upstream 节点依此发送 InitFC1-P、InitFC1-NP 和 InitFC1-Cpl[⊖] 报文初始化 Upstream 节点使用的 Credit。

此时 Current 节点还可能收到来自 Downstream 节点的 InitFC1-P、InitFC1-NP 和 InitFC1-Cpl 报文，并初始化 Current 节点的 Credit。当 Current 节点收到这些来自 Downstream 节点的 FCP 后，将设置对应缓冲的 FI1 状态位为 1。

当 Current 节点所有缓存的 FI1 状态位有效后，VC 将进入 FC_INIT2 状态。Current 节点进入 FC_INIT2 状态之前，Upstream 节点获得的 Credit 和 Current 节点的空余缓存大小相等，Current 节点获得的 Credit 和 Downstream 节点的空余缓存大小相等。这一点和其他流量控制机制类似，即 Crd_Bal 的初始值为 Buf_Alloc。

PCIe 总线还提供了 FC_INIT2 状态，该状态的主要功能是验证 FC_INIT1 的结果。当节点进入 FC_INIT2 状态时，与流量控制相关的缓存已经初始化完毕。PCIe 总线设置 FC_INIT1 和 FC_INIT2 这两个状态与数据链路层的状态机相关。

如第 7.1.1 节所示，当数据链路层处于 DL_Init 状态时，将初始化 PCIe 总线的流量控制机制。当 VC 处于 FC_INIT1 状态时，数据链路层通知事务层 DL_Down 状态位有效，此时事务层不能向对端设备发送 TLP，从而流量控制机制的初始化可以在一个“相对没有干扰的环境”下进行。

而当 Current 节点的 VC 进入 FC_INIT2 状态时，事务层需要首先禁止使用这条 VC 发送报文，之后 Current 节点向 Upstream 节点依此发送 InitFC2-P、InitFC2-NP 和 InitFC2-Cpl 报文初始化 Upstream 节点的发送缓冲。当 Upstream 节点收到这些报文之后，将丢弃这些报文中包含的 Credit 信息，并设置相应的 FI2 状态位。

同理 Current 节点也将收到来自 Downstream 节点的 InitFC2-P、InitFC2-NP 和 InitFC2-Cpl

⊖ PCIe 总线规定每隔 34 μs 发送一组 InitFC 报文。

报文，并设置 Current 节点的 FI2 状态位。当所有数据缓存的 FI2 状态位有效后，将完成 PCIe 链路流量控制机制的初始化。最后数据链路层通知事务层 DL_Active 状态位有效，此时事务层可以使用这个 VC 发送 TLP。

9.3.4 PCIe 设备如何使用 FCP

PCIe 总线完成流量控制的初始化之后，Current 节点、Upstream 节点和 Downstream 节点通过发送 UpdateFC-P、UpdateFC-NP 和 UpdateFC-Cpl 报文进行流量控制。

1. Upstream 节点向 Current 节点发送报文

Upstream 节点向 Current 节点发送报文时，需要设置一些参数。

1) CREDITS_CONSUMED，简称为 CC。该参数存放 Upstream 节点已经发送了多少个 FC Unit，不同数据缓存的 FC Unit 的大小见表 9-3。在 PCIe 设备初始化时该参数为 0，之后 PCIe 设备每发送一个 FC Unit，该值将加 1。PCIe 总线规定

$$CC = (CC + \text{Increment}) \bmod 2^{\text{Field Size}}$$

其中 Increment 指发送的 TLP 含有的 FC Unit 个数。其中 PH、NPH 和 CplH 的 Field Size 参数为 8，而 PD、NPD 和 CplD 的 Field Size 参数为 12。

2) CREDIT_LIMIT，简称为 CL。该参数存放当前节点的 Credit，在 VC 初始化时，该参数通过收到的 InitFC1 报文赋值。此后每收到 UpdateFC 报文时，Current 节点将此参数与 UpdateFC 报文中的 Credit 比较，如果结果不等，则使用 UpdateFC 报文中的 Credit 对此参数重新赋值。

在 Upstream 节点中，设置了一个 Credit 检查逻辑（Transmitter Gating Function），用来判断在 Current 节点中是否有足够的缓存接收即将发送的 TLP。如果 Upstream 节点没有足够的 Credit 时，则不能向对端设备发送这个 TLP。

Upstream 节点检查缓存的算法如下。首先将 CUMULATIVE_CREDITS_REQUIRED 参数简称为 CR，而 $CR = CC + \text{PTLP}$ （即将发送 TLP 所需要的 Credit）。当以下公式

$$(CL - CR) \bmod 2^{\text{Field Size}} \leq 2^{\text{Field Size}} / 2$$

成立时，表示 Upstream 节点可以向 Current 节点发送 TLP 报文。

使该公式成立有一个额外需求，即每次发送的 Credit 小于 $2^{\text{Field Size}} / 2$ ，此时 CL 不可能比 CC 大 $2^{\text{Field Size}} / 2$ 。当 $(CL - CR) \bmod 2^{\text{Field Size}}$ 不大于 $2^{\text{Field Size}} / 2$ 时，表示 Current 节点有足够的缓冲接收来自 Upstream 节点的报文；如果 $(CL - CR) \bmod 2^{\text{Field Size}}$ 大于 $2^{\text{Field Size}} / 2$ 时，运算结果是一个负数，表示 Current 节点没有足够的缓冲接收来自 Upstream 节点的报文。

2. Current 节点从 Upstream 节点接收报文

Current 节点接收来自 Upstream 节点发送报文时，也需要设置一些参数。

1) CREDITS_ALLOCATED，简称为 CA。该参数存放 Current 节点一共允许 Upstream 节点发送多少个 FC Unit，不同数据缓存的 FC Unit 初始化后使用的最小值见表 9-3。在初始化时该参数为 Current 节点接收缓冲的大小，之后 Current 节点每分配一个 FC Unit，该值将

○ 相当于 N23 算法的 N3 等于 $2^{\lceil \text{Field Size} \rceil} / 2$ ，此时可以防止 Current 节点的 Underrun。但是 PCIe 总线并没有规定发送 Credit 的频率。

加 1。PCIe 总线规定：

$$CA = (CA + \text{Increment}) \bmod 2^{\text{Field Size}}$$

其中 Increment 指 Current 节点新分配的 FC Unit 个数。PH、NPH 和 CplH 的 Field Size 参数为 8，而 PD、NPD 和 CplD 的 Field Size 参数为 12。Current 节点使用 UpdateFC DLLP 将 CA 传递给 Upstream 节点，之后 Upstream 节点使用该值更新 CL 参数。

2) CREDIT_RECEIVED，简称为 CRCV。该参数存放 Current 节点一共接收了多少个 FC Unit。在初始化时该参数为 0，之后 Current 节点每接收一个 FC Unit，该值将加 1。PCIe 总线规定：

$$\text{CRCV} = (\text{CRCV} + \text{Increment}) \bmod 2^{\text{Field Size}}$$

其中 Increment 指 Current 节点新接收的 FC Unit 个数。

Current 节点可以设置一个逻辑检查部件，检查来自 Upstream 节点的 TLP 报文是否会造成 CA 的溢出。这个逻辑检查部件是一个可选件，因为 Upstream 节点在发送 TLP 时，已经保证了 Current 节点的 CA 不会溢出。当以下公式

$$(CA - \text{CRCV}) \bmod 2^{\text{Field Size}} \geq 2^{\text{Field Size}} / 2$$

成立时，Current 节点认为 CA 溢出，此时 Current 节点将抛弃来自 Upstream 节点的 TLP，而并不改变 CRCV 的值，同时释放为这个 TLP 预先分配的缓存空间。

9.4 小结

本章仅使用了较少的篇幅讲述 PCIe 总线的流量控制机制，而重点讨论通用流量控制机制的基本原理。PCIe 总线由于强调与 PCI 总线的兼容，流量控制机制的设计并不完美。PCIe 总线的主要应用领域依然是 PC，在这个领域中，流量控制并不是最重要的。